

Batched Interventions

Contents

1. Motivation	1
2. Goals & requirements	2
3. Key design decision: where-semantics, not split	2
4. Mechanism	3
5. API	4
6. Semantics specification	5
7. Status of the current PR & corrections	5
8. Implementation stages	6
9. Open questions	6
10. Progress log	7

Status: **draft / under design** — companion to PR [#594](#).

1. Motivation

In search for explanation (and many other causal workflows), we need to evaluate **large batches of interventions**. These interventions are *heterogeneous* — different elements of the batch may intervene on different variables, with different values.

ChiRho already provides two counterfactual handlers living at two edges of a spectrum:

- **TwinWorldCounterfactual (TWC)** — factual world plus **one** alternative world. All splits reuse a single shared dimension name, so the world axis has size 2. Computationally/memory-wise cheap, but one can track only one counterfactual scenario.
- **MultiWorldCounterfactual (MWC)** — factual world plus **every** combination of interventions. Each split allocates its **own** index-plate dimension, so the represented worlds are the full **Cartesian product** of all interventions. Memory grows **exponentially** in the number of intervened sites.

A *batched* intervention handler is the natural third point on this spectrum: **many alternative scenarios laid side-by-side on a single shared axis**. This gives access to many coordinated interventions without the compute and memory cost of materializing every *combination* of them — combinations that, in practice, are usually never inspected and are discarded anyway.

2. Goals & requirements

1. **Linear memory** in the batch size N — a single shared world axis (conceptually, TWC scaled to $N+1$ worlds).
2. **Faster than explicitly looping** (with do or TWC) over the interventions — one vectorized forward pass instead of N separate model evaluations.
3. **Correct downstream propagation**
4. **An always present factual world**, addressable like TWC/MWC via `gather(value, IndexSet(<name>={0}))`.
5. **Works with non-trivial event dimensions** (`to_event(k)`).
6. **Citizen status alongside TWC/MWC** — a Messenger class usable like the other handlers (as a context manager or decorator), composing with `FactualConditioningMessenger`, whose outputs are read with the standard `chirho.indexed` operations (`gather`, `IndexSet`, `indices_of`).
7. **Less memory and faster than MWC or looped TWC**

3. Key design decision: where-semantics, not split

A key question was whether the batched handler should route through the existing `split` primitive (as TWC/MWC do) with a shared dimension name, or use a `torch.where(mask, act, obs)` mechanism. Prototyping showed that `split` is the wrong primitive here.

Recall how `split` builds worlds. `split(obs, acts, name)` puts the observed value at index 0 (the factual world) and, for each alternative `act_i`, stores `intervene(factual, act_i)` at index $i+1$. Every counterfactual world is rebuilt from the factual, index-0 value, which breaks things if one wants to keep track of counterfactual worlds on a single axis.

On a single shared axis, suppose a scenario intervenes on z but leaves y alone, in a model where y depends on z . By the time the handler reaches y , that scenario's slot on the shared axis already holds a value of y that has propagated the upstream z intervention. Leaving y alone should keep that propagated value — but `split` rebuilds the slot from the index-0 factual y (the world where z was never touched), erasing the propagation.

The proposed solution is to use `torch.where(mask, act, obs)`. This, by contrast, keeps `obs` wherever a scenario does not intervene, which is exactly that propagated value.

Concretely, as an example, for the model $z \rightarrow \{x, y\}$, $y \leftarrow 0.8x + 0.3z$, with scenario $0 = \{z: 100\}$ (and y untouched):

- **Ground truth (MWC, `gather(z={1}, y={0})`):** $z = 100$, $y = 108.66$ (the $z = 100$ intervention propagates into y).
- **Shared-name `split` prototype:** $z = 100$, $y = 0.06$ — y was reset to the factual value. Wrong.
- **`torch.where` (original PR mechanism):** $z = 100$, $y \approx 109$ — propagation preserved. Desired.

The reason MWC gets propagation right is because it gives each site its own axis. This, however, causes the memory blow-up in larger models with non-trivial explanation search.

3.1 Noise semantics: shared vs. independent (shared_noise flag)

A second, orthogonal decision concerns the *exogenous noise* of latent sites. The question is whether we draw `BatchedLatents` over all latents, giving each scenario an independent draw. This is a modeling decision that should be left to the user. Holding noise **fixed** across worlds (true counterfactuals) is achieved by **not** batching the latents and instead letting the intervention's where create the shared axis at intervened sites and propagate it downstream — exactly as MWC does. Verified to be **bit-for-bit MWC-equivalent** (including `event_shape` $\in \{(), (4,), (4, 3)\}$).

Both are supported via a **shared_noise flag (default True)**:

- `shared_noise=True` (default): drop `BatchedLatents`; exogenous noise shared; MWC-equivalent; linear memory. The `...Counterfactual` name applies here.
- `shared_noise=False`: keep `BatchedLatents`; independent per-scenario draws; batched interventional Monte Carlo.

4. Mechanism

Two cooperating pieces; the design corrects how they are wired together.

4.1 The named batch dimension

The single shared world axis is a **named** index-plate dimension of size $N+1$ tracked by `chirho.indexed`. With `shared_noise=True` (default) it is created lazily by the intervention itself (Section 4.2) at the first intervened site and propagates downstream, leaving upstream latents shared/scalar. With `shared_noise=False` it is instead created eagerly on *every* latent site by `BatchedLatents(N+1, name=...)`, which already handles event dimensions correctly (a `to_event(1)` latent is placed at the registered plate position; consumers must pass `event_dim` to `gather/indices_of`).

4.2 Masked replacement placed on the named dimension

For each intervened site, the handler computes

```
value = torch.where(mask, act, obs)
```

where, **crucially**, `act` and `mask` are first moved onto the named batch dimension with

```
act = unbind_leftmost_dim(act, name, size=N+1, event_dim=event_dim)
mask = unbind_leftmost_dim(mask, name, size=N+1, event_dim=event_dim)
```

`event_dim` is the site's event dimensionality, which already flows through `msg["kwargs"]` from `Interventions.pyro_post_sample(intervene(..., event_dim=len(fn.event_shape), name=...))`.

This is the **one fix** the original PR is missing. Without it, `act/mask` keep their batch on a raw leftmost axis that does not coincide with the named plate dimension, which causes three observed failures:

- a spurious **duplicate** batch axis $\rightarrow$ memory $O(N^2)$ instead of $O(N)$;
- gather/IndexSet fail to isolate scenarios (the named axis is not the one holding the values);
- a hard **crash** on sites with non-trivial event dimensions (RuntimeError: size of tensor a (N) must match ... b (D)).

With the fix, all three are resolved (verified on `to_event(1)` models: `gather(IndexSet(name={i}), event_dim=1)` isolates scenario `i`).

4.3 The factual world

Reserve **index 0** of the batch axis as the factual world: an extra batch slice whose mask is False for *every* site, so the factual value propagates through it unchanged. Scenarios occupy indices `1..N`. This matches the TWC/MWC convention (`gather(value, IndexSet(name={0}))` is the factual world) and, unlike `split`, composes naturally with where-semantics (an all-False mask column is simply the identity).

5. API

5.1 Handler class (citizen alongside TWC/MWC)

```
class BatchedWorldCounterfactual(IndexPlatesMessenger):
    """Factual world + N heterogeneous intervention scenarios on one shared axis."""
```

- Subclasses `IndexPlatesMessenger` and, in `__enter__/_exit__`, enters a small stack of inner handlers: `_BatchedInterventions` (the masked where placement); `BatchedLatents` **only** when `shared_noise=False`; and `FactualConditioningMessenger` when `factual=True`. It deliberately does **not** inherit `BaseCounterfactualMessenger`, because that routes intervene through `split`, whose reset-to-factual-0 semantics are incorrect on a shared axis (Section 3).
- Self-contained: allocates its own index plates (no externally supplied `IndexPlatesMessenger` required — the previous `batched_do` required one).
- The two accepted input forms are captured by the public type alias `:data:BatchedInterventions`; `_build_batched_actions` lowers either to the internal `{site: _BatchedAction}` representation.
- Usable as both a context manager and a decorator (parity with TWC/MWC).

5.2 batched_do convenience wrapper

A context manager / wrapper that constructs the handler from either input form:

1. **Collection of do-style dicts** — `[{site: act}, ...]`. Actions restricted to tensors; broadcast together. Element `i` is scenario `i`.
2. **Mapping {site: (act, mask)}** — `act` of shape `(N, ...)`, `mask` of shape `(N,)`, giving explicit per-element control.

Both lower to the internal `{site: _BatchedAction(act, mask)}` representation, plus the reserved factual column at index 0.

6. Semantics specification

For batch size N , world axis of size $N + 1$:

- **Index 0** — factual world; no interventions applied.
- **Index $i+1$** — scenario i : for each site s , $\text{value}[i+1] = \text{act}_{\{s,i\}}$ if scenario i intervenes on s , else the propagated $\text{obs}[i+1]$ (i.e. the value implied by scenario i 's *other* interventions; factual if it has none upstream).
- Reading results: `gather(value, IndexSet(<name>={k}), event_dim=...)` selects world k ; `indices_of(value, event_dim=...)` reports the populated axis.

Equivalence property (for validation). With `shared_noise=True`, a batched world equals the corresponding gathered MWC slice. The equality is **sample-path exact** when the batched axis and the MWC axes consume the random-number stream identically — i.e. for single-site alternatives (`test_matches_mwc_single_site`, parametrized over `event_shape` and `first_available_dim`) and for interventions on terminal/leaf sites (`test_matches_mwc_multisite_leaf`). For *multi-site* interventions with **sampled descendants**, the batched single axis (size $N+1$) and MWC's Cartesian axes have different shapes, so a shared-noise downstream sample draws different noise per cell: the worlds are then **distributionally** equivalent but not sample-path identical. The single-axis structure itself (no cross product) is checked by `test_multiple_interventions_stay_on_single_axis` and `test_aligned_multisite_interventions`.

7. Status of the current PR & corrections

Found and addressed while reviewing PR #594:

- **Fixed (committed):** the “collection of dicts” path was broken — the `batched_acts` comprehension iterated the leftover loop variable instead of the accumulated acts, and used `torch.broadcast_tensors(v)` instead of `*v`; `batch_size` consequently collapsed to 1 and silently produced wrong results. Also removed a dead name parameter on `_BatchedInterventions`.
- **Done (Stage 1):** routed `act/mask` through `unbind_leftmost_dim(..., event_dim=...)` (Section 4.2) so the batch lands on the named plate. Resolves the duplicate-axis ($O(N^2)$), broken-gather, and event-dim-crash failures.
- **Done (Stage 2):** reserve index 0 of the batch axis as the factual world via `_prepend_factual_world` (all-False mask column), gated by a factual flag on `batched_do` (default True); scenarios shift to indices $1..N$.
- **Done (Stage 3):** added the self-contained `BatchedWorldCounterfactual Messenger` (subclasses `IndexPlatesMessenger`; internally composes the inner handlers). No enclosing `IndexPlatesMessenger` is required; `batched_do` is now a thin wrapper.
- **Done (Stage 4):** composed `FactualConditioningMessenger` (when `factual=True`) so conditioning applies observed data only to the factual world, as in MWC/TWC.
- **Done (noise semantics):** added a `shared_noise` flag (default True). True holds exogenous noise fixed across worlds (drops `BatchedLatents`), making the handler **exactly MWC-equivalent** on a single linear-memory axis; False keeps `BatchedLatents` for independent per-scenario draws (batched interventional Monte Carlo). See Section 3.1.
- **Done (Stage 5):** MWC-equivalence and analogous TWC/MWC tests, in a dedicated module `tests/interventional/test_batched_interventions.py`.

8. Implementation stages

1. **[done] Correctness fix for the existing `batched_do`** — place `act/mask` on the named dimension via `unbind_leftmost_dim` with `event_dim`; tests for gather isolation, event dimensions, and linear memory (`test_gather_isolates_scenarios_and_propagates`, `test_event_dim_collection_form`, `test_event_dim_mapping_form`).
2. **[done] Factual world** — reserve index 0 (`_prepend_factual_world`, `factual=True` by default); tests against `gather(name={0})` (`test_factual_world_present_and_optional`, plus the index shift verified across the suite).
3. **[done] `BatchedWorldCounterfactual` handler class** — self-contained index plates; `batched_do` is now a thin constructor wrapper (`test_factual_world_present_and_optional` and `test_handler_usable_as_decorator` exercise the class directly).
4. **[done] `FactualConditioningMessenger` composition** — observe-after-intervene parity with TWC/MWC (`test_conditioning_applies_to_factual_world`).
5. **[done] MWC-equivalence test suite** — `batched` scenarios match the corresponding gathered MWC slices, parametrized over `event_shape` and `first_available_dim` (`test_matches_mwc_single_site`, `test_multiple_interventions_stay_on_single_axis`).
6. **[done] Benchmarks.** Two deterministic guards run in CI: `test_linear_memory_vs_mwc_cross_product` (element counts: MWC 2^K vs. `batched` $N+1$) and `test_batched_evaluates_model_once` (one vectorized model evaluation vs. N for an explicit loop). Two wall-clock/allocation benchmarks are runnable locally but `@pytest.mark.skip`-ped in CI (matching `tests/robust/test_performance.py`):
 - `test_benchmark_batched_vs_alternatives` — evaluates N single-site alternatives four ways. Measured (single thread, $N=300$, `event_size=300`): **batched ~1.6 ms**, `loop+do` ~30 ms (~19x), `loop+TwinWorldCounterfactual` ~128 ms (~80x), `MultiWorldCounterfactual` ~19 ms (~12x). `Batched` beats even MWC here, as MWC carries more per-site machinery (split/scatter + factual conditioning).
 - `test_benchmark_batched_memory_vs_mwc` — bytes of the represented worlds as the number of intervened sites K grows; ratio $2^K/(K+1)$. Measured (float64): $K=4$ ~3x, $K=8$ ~28x, **$K=12$ ~315x** (16384 vs. 52 bytes).

9. Open questions

- **Naming:** `BatchedWorldCounterfactual`? `BatchedInterventionCounterfactual`? (Current: `BatchedWorldCounterfactual`.)
- **Optional factual world:** *Resolved:* index 0 is materialized by default (`factual=True`); a `factual=False` flag drops it for pure scenario-vs-scenario comparison.
- **Mixed dtypes / masked placeholders:** *Resolved (verified):* the collection path coerces a site's scenarios to a common dtype — when a masked slot uses the nan float placeholder the column promotes to float; when every scenario intervenes (no placeholder) the dtype is preserved. The `(act, mask)` mapping form always preserves the user's dtype exactly (no placeholder), so discrete/integer sites needing dtype control should use that form.
- **Interaction with model-internal plates** — the $N+1$ axis multiplies with any existing batch/plate dims (true of all indexed handlers); document expected shapes.

10. Progress log

2026-06-23

Branched ru-batched-intervention off PR #594 (jf-batched-intervention), merged latest master (resolved conflicts toward master's modernized typing).

Done today:

- **Bug fixes** in the original `batched_do`: the `collection-of-dicts` path silently produced wrong results (iterated the wrong loop variable; `batch_size` collapsed to 1); removed a dead `name` param.
- **Stage 1** — `act/mask` now placed on the named index-plate `dim` via `unbind_leftmost_dim(..., event_dim=...)`. Fixes gather isolation, the $O(N^2)$ duplicate-axis memory, and the event-dim crash.
- **Stage 2** — factual world materialized at index 0 (`_prepend_factual_world`, `factual=True` default; `factual=False` drops it).
- **Stage 3** — self-contained `BatchedWorldCounterfactual(IndexPlatesMessenger)`; `batched_do` is now a thin wrapper. No enclosing `IndexPlatesMessenger` needed.
- **Stage 4** — `FactualConditioningMessenger` folded in (when `factual=True`): conditioning applies observed data to the factual world only.
- **Noise semantics** — settled the key question: `shared_noise=True` (default) drops `BatchedLatents` → **bit-for-bit MWC-equivalent**, linear memory; `shared_noise=False` → independent draws (batched interventional MC).
- **Stage 5** — tests moved to dedicated `tests/interventional/test_batched_interventions.py` (32 tests): analogues to the TWC/MWC suite (smoke, MWC-equivalence over `event_shape/first_available_dim`, single-axis vs. cross-product, conditioning, dim-allocation failure) plus batch-specific tests (mapping/(`act`, `mask`) form, event dims, broadcasting, `factual` on/off, `shared` vs. `independent` noise, linear-memory-vs-MWC, error cases). All green; lint clean; no regressions in the broader suite.
- Also confirmed the 24 `xfailed` in `test_counterfactual_handler.py` are a pre-existing `MWC↔infer_discrete` enumeration bug (`num_steps=5`), unrelated to this work and correctly neutralized by `xfail`.

2026-06-23 (continued)

- **Stage 6 benchmarks** — added `test_batched_evaluates_model_once` (deterministic, in CI) and the skipped `test_benchmark_batched_vs_alternatives` (batched vs. `loop+do` vs. `loop+TWC` vs. MWC) and `test_benchmark_batched_memory_vs_mwc`.
- **Refactor** of `chirho/interventional/handlers.py`: `public :data:BatchedInterventions` type alias (removes the 3x-duplicated union), `_DEFAULT_BATCH_NAME` constant, `_BatchedAction.batch_size` property, full type hints, and Sphinx docstrings (no inline types). Behaviour unchanged.
- **Coverage gaps closed** (the three identified against this doc): multi-site leaf MWC-equivalence (`test_matches_mwc_multisite_leaf`), decorator usage (`test_handler_usable_as_decorator` and `shared_noise=False` with event dims (`test_independent_noise_event_dim`)). Also added the canonical aligned/zipped multi-site case (`test_aligned_multisite_interventions`).
- **Clarified §6**: `shared-noise` equivalence to MWC is sample-path exact only when axis shapes match (single-site or leaf-multisite); multi-site with sampled descendants is distributionally (not sample-path) equivalent.
- Test module now at 38 passing + 2 skipped benchmarks; lint clean.

Still open:

- Wire this doc into `design_notes/index.rst` (convert to `.rst`); decide on pushing the branch / updating PR #594.
- Handler **naming** (currently `BatchedWorldCounterfactual`).
- Document **model-internal plate** interaction (the $N+1$ axis multiplies with existing batch/plate dims).
- Optional inference-compatibility smoke tests (SVI/Predictive) for MWC parity.